


# CLOUD-NATIVE FORENSICS

AWS & Azure Triage, Kubernetes Audit,
& CNAPP Architecture.

---

Disusun Oleh: **Rizki Angga Pratama**
L2 SOC Analyst

# HAK CIPTA & DISCLAIMER

**Cloud-Native Forensics & CNAPP Architecture**
Edisi Enterprise (50 Halaman)

Disusun oleh: Rizki Angga Pratama, S. Kom, C. Pro

Dokumen ini adalah cetak biru teknis bagi *Cloud Security Engineers* dan Tim Respons Insiden (IR) yang bertugas melindungi serta menginvestigasi serangan siber di lingkungan infrastruktur awan modern (AWS, Azure, Kubernetes, dan Serverless).

> **Ancaman Ephemeris:**
> Di dalam komputasi awan, infrastruktur bersifat *ephemeral* (sementara). Sebuah kontainer *Docker* atau instans *AWS EC2* yang terinfeksi malware bisa dihapus otomatis oleh sistem *Auto-Scaling* dalam hitungan menit. Pendekatan forensik tradisional (seperti membawa perangkat keras ke lab) tidak berlaku di sini. Kecepatan dan agregasi *API Log* adalah segalanya.

**Filosofi Desain:**
Buku ini disajikan dalam tema *Nebula Blue & Serverless Azure*. Biru siber melambangkan kedalaman lautan data telemetri awan, arsitektur mikroservis tanpa batas, dan ketangkasan operasi perlindungan *cloud-native*.

© 2026 Hak cipta dilindungi undang-undang. Panduan Investigasi Awan Enterprise.

# KATA PENGANTAR

Pergeseran beban kerja (workloads) perusahaan dari pusat data fisik (On-Premise) menuju komputasi awan (AWS, Azure, GCP) dan orkestrasi kontainer (Kubernetes) telah mengubah seluruh paradigma keamanan siber.

Di lingkungan fisik, peretas harus menembus *Firewall*, mengeksploitasi *server*, dan membuang waktu berminggu-minggu untuk menyebar ke jaringan lain. Di lingkungan awan, jika seorang pengembang tak sengaja mengunggah (commit) Kunci API AWS (*Access Key*) ke GitHub publik, peretas dapat menguasai seluruh perusahaan Anda, menghapus miliaran data database S3, dan menyalakan ribuan *server penambang kripto (Cryptomining)* dari benua lain—semuanya hanya dalam waktu **kurang dari 3 menit**, tanpa pernah sekalipun menyentuh firewall Anda.

Untuk bertahan di medan perang Cloud-Native ini, analis keamanan tidak bisa lagi hanya mengandalkan Antivirus (EDR). Anda harus menguasai bahasa **Control Plane**. Anda harus memahami Identity and Access Management (IAM), menganalisis *AWS CloudTrail* dan *Azure Activity Logs* menggunakan kueri SQL/KQL, dan memahami cara mengisolasi Pod Kubernetes yang telah disusupi secara terprogram (API-driven response).

Buku 50 halaman ini adalah manual investigasi awan Anda. Dari arsitektur CNAPP hingga forensik *serverless*, kita akan membedah anatomi serangan *Cloud* modern dan bagaimana melawannya dengan otomatisasi.

*— Rizki Angga Pratama*

*Author*

# DAFTAR ISI (1/2)

## BAGIAN I: Paradigma Awan & Arsitektur CNAPP


Bab 1.1 Tradisional DFIR vs Cloud-Native DFIR Hlm. 7

Bab 1.2 Model Tanggung Jawab Bersama (Shared Responsibility) Hlm. 8

Bab 1.3 Anatomi CNAPP (CSPM, CWPP, CIEM) Hlm. 9

Bab 1.4 The Cloud Kill Chain (MITRE ATT&CK for Cloud) Hlm. 10

Bab 1.5 Ephemeral Workloads & Tantangan Akuisisi Memori Hlm. 11


## BAGIAN II: AWS Forensics & CloudTrail Triage


Bab 2.1 Visibilitas AWS: CloudTrail, VPC Flow, GuardDuty Hlm. 13

Bab 2.2 Berburu Anomali IAM menggunakan Amazon Athena (SQL) Hlm. 14

Bab 2.3 SSRF Attacks & Pencurian Kredensial IMDSv1/v2 Hlm. 15

Bab 2.4 Investigasi Ransomware di Amazon S3 Hlm. 16

Bab 2.5 Snapshot Forensics: Mengakuisisi Volume EBS EC2 Hlm. 17

Bab 2.6 Mengekstrak Jejak Jaringan (VPC Flow Logs) Hlm. 18

Bab 2.7 Analisis Keterlibatan Akun Pihak Ketiga (Cross-Account) Hlm. 19

# DAFTAR ISI (2/2)

## BAGIAN III: Azure DFIR & Entra ID (Azure AD)


Bab 3.1 Lanskap Log Azure (Activity Log, NSG Flow) Hlm. 21

Bab 3.2 Triage Log Azure AD (Otentikasi & Kondisional) Hlm. 22

Bab 3.3 Serangan Illicit Consent Grant (OAuth Phishing) Hlm. 23

Bab 3.4 Golden SAML & Perburuan Federasi Kredensial Hlm. 24

Bab 3.5 Berburu Ancaman menggunakan Microsoft Sentinel (KQL) Hlm. 25

Bab 3.6 Mengisolasi Azure Virtual Machines (Network Security Groups) Hlm. 26


## BAGIAN IV: Kubernetes (K8s) & Container Forensics


Bab 4.1 Permukaan Serangan Klaster Kubernetes Hlm. 28

Bab 4.2 Triage K8s API Audit Logs (Mendeteksi Admin Liar) Hlm. 29

Bab 4.3 Anatomi Container Escape (Privileged Pods & Mounts) Hlm. 30

Bab 4.4 Penyalahgunaan Docker Socket (docker.sock) Hlm. 31

Bab 4.5 Teknik Akuisisi Memori Pod (Sidecar Pattern) Hlm. 32

Bab 4.6 Membekukan Kontainer (Cordon & Taint / Pause) Hlm. 33


## BAGIAN V: Serverless & Automated Incident Response


Bab 5.1 Lanskap Forensik Nirserver (AWS Lambda) Hlm. 35

Bab 5.2 Mendeteksi Backdoor pada Fungsi Lambda Hlm. 36

Bab 5.3 Automated Containment Architecture (EventBridge & SOAR) Hlm. 37

Bab 5.4 Playbook: Otomasi Pencabutan Kredensial (STS Revoke) Hlm. 38

Bab 5.5 Playbook: Otomasi Isolasi Instans EC2 Terinfeksi Hlm. 39

Bab 5.6 Remediasi Postur Berkelanjutan (Shift-Left Security) Hlm. 40

Glosarium Cloud Security (A-M) Hlm. 48

Glosarium Cloud Security (N-Z) Hlm. 49

Back Cover (The Elastic Perimeter) Hlm. 50




# BAGIAN I

Paradigma Awan &
Arsitektur CNAPP

# BAB 1: PARADIGMA OPERASIONAL

## 1.1 Tradisional DFIR vs Cloud-Native DFIR

Digital Forensics & Incident Response (DFIR) tradisional mengandalkan kontrol fisik. Saat *server* diretas, analis akan datang secara fisik (atau via *vSphere*), mematikan jaringan (*Pull the plug*), lalu melakukan pencitraan (*Imaging*) *hard disk* Bit-by-Bit menggunakan alat seperti FTK atau EnCase untuk dianalisis berbulan-bulan.

**Cloud-Native DFIR** menghancurkan konsep tersebut. Anda tidak memiliki akses fisik ke *server* AWS/Azure. IP berubah-ubah (*Dynamic IP*). Bahkan infrastruktur sasaran (sebuah Kontainer *Docker*) mungkin hanya hidup selama 3 menit sebelum mesin orkestrasi menghapusnya untuk selamanya (*Ephemeral Workloads*).

Di *Cloud*, fokus investigasi bergeser dari **Forensik Sistem Operasi** menuju **Forensik Control Plane (API)**. Peretas awan tidak perlu menjatuhkan *malware* ke *hard disk*. Mereka hanya membutuhkan Kunci Akses (*IAM API Keys*) untuk menyuruh infrastruktur awan Anda menghapus dirinya sendiri.

## 1.2 Shared Responsibility Model

Kegagalan memahami **Model Tanggung Jawab Bersama** adalah penyebab utama kebocoran data di perusahaan. AWS, Azure, dan GCP mengamankan *Cloud itu sendiri* (Perangkat keras, fisik pusat data, jaringan listrik, dan keamanan *Hypervisor*).

Namun, pelanggan (Perusahaan Anda) bertanggung jawab atas keamanan **DI DALAM Cloud**.

> **Garis Batas Investigasi:**
> Jika kerentanan ada di hipervisor AWS (yang mana sangat jarang), itu adalah tugas Amazon. Tetapi, jika sebuah *Bucket S3* bocor ke internet publik karena *Engineer* Anda menyetel izin (*Permissions*) menjadi `"Public Read"`, AWS tidak akan bertanggung jawab atas data yang dicuri. Konfigurasi, Manajemen Akses (IAM), dan enkripsi data adalah tanggung jawab 100% dari Tim Keamanan Anda.

Saat merespons insiden (IR), jangan membuang waktu meminta AWS untuk memberikan *log* fisik *server* mereka. Anda harus sudah mengumpulkan log aplikasi awan Anda (CloudTrail) secara mandiri ke keranjang S3 forensik yang terpisah (*Isolated Log Archive*).

## 1.3 Anatomi CNAPP

Karena arsitektur awan sangat kompleks (Kombinasi VM, Kontainer, Database, dan Serverless), industri melahirkan platform keamanan super yang disebut **Cloud-Native Application Protection Platform (CNAPP)**. CNAPP adalah penggabungan tiga pilar keamanan:

- **CSPM (Cloud Security Posture Management):** Memindai kesalahan konfigurasi infrastruktur. (Contoh: Memberi peringatan jika *Port 22/SSH* terbuka ke `0.0.0.0/0` di *Security Group* AWS, atau mendeteksi *Bucket S3* yang tidak dienkripsi).
- **CWPP (Cloud Workload Protection Platform):** Bertindak seperti EDR untuk *Cloud*. Memindai kerentanan di dalam citra (*Images*) Docker, mendeteksi eksploitasi memori di *Runtime*, dan memblokir *malware* di dalam OS Linux EC2.
- **CIEM (Cloud Infrastructure Entitlement Management):** Fokus pada IAM. Mendeteksi identitas pengguna atau akun layanan (*Service Account*) awan yang memiliki izin (*Privileges*) berlebihan yang tidak pernah mereka gunakan (*Over-provisioned/Least Privilege violation*).

## 1.4 The Cloud Kill Chain

Peretas awan (*Cloud Threat Actors*) beroperasi dengan urutan serangan yang berbeda dari peretas jaringan internal. *MITRE ATT&CK for Cloud* memetakan perilaku khusus ini.




Serangan jarang bermula dari eksploitasi kode peladen (*Remote Code Execution/RCE*). Sebanyak 80% insiden awan bermula dari Kredensial IAM yang bocor di Repositori GitHub publik atau kredensial curian akibat *Phishing* (*Initial Access*). Musuh kemudian memanggil API `sts:AssumeRole` untuk meningkatkan haknya, lalu menyalakan ribuan *server* (EC2) mahal untuk menambang kripto atas tagihan kartu kredit perusahaan Anda (*Resource Hijacking*).

## 1.5 Ephemeral Workloads & Tantangan Memori

Bayangkan Anda menerima peringatan (*Alert*) bahwa kontainer Kubernetes `web-frontend-pod-1a2b` menunjukkan koneksi jaringan ke IP *malware* Rusia. Saat analis Anda masuk 5 menit kemudian, *Pod* tersebut sudah lenyap dari *cluster* karena fitur *Auto-Scaling* menurunkan skalanya akibat lalu lintas web yang mereda.

Jejak *malware*, skrip yang dieksekusi di memori RAM, dan sejarah berkas yang dimodifikasi hancur selamanya. Ini adalah tantangan terbesar DFIR *Cloud-Native*.

Solusi arsitekturalnya adalah mengimplementasikan agen keamanan CWPP tingkat-*Kernel* (menggunakan *eBPF - Extended Berkeley Packet Filter*). Agen *eBPF* merekam seluruh panggilan sistem (*Syscalls*)—seperti pembuatan berkas, pembukaan jaringan, dan eksekusi memori—dari dalam tingkat terdalam sistem operasi, lalu mengalirkannya ke *Database* Eksternal (SIEM) secara *Real-Time*. Meskipun kontainer terhapus, bukti forensik kelakuannya tertinggal secara permanen di SIEM Anda.




# BAGIAN II

AWS Forensics & CloudTrail Triage
(Investigasi Control Plane)

# BAB 2: AMAZON WEB SERVICES (AWS)

## 2.1 Visibilitas: CloudTrail, VPC Flow, GuardDuty

Di AWS, jika Anda tidak menyalakan *Logging*, Anda terbang buta (*Flying Blind*). Ada tiga pilar visibilitas yang mutlak wajib dihidupkan oleh setiap perusahaan:

1. **AWS CloudTrail (The Control Plane):** Mencatat **SETIAP** panggilan API yang dilakukan di akun AWS Anda. Siapa yang menghidupkan *server*? Siapa yang menghapus basis data? Siapa yang mengubah konfigurasi *Firewall*? Semuanya dicatat di sini. Ini adalah cawan suci (*Holy Grail*) forensik AWS.
2. **VPC Flow Logs (The Network):** Setara dengan *NetFlow* pada *Router* fisik. Mencatat metadata lalu lintas jaringan (IP Asal, IP Tujuan, Port, Byte yang dikirim) yang masuk dan keluar dari *Virtual Private Cloud (VPC)* Anda. Sangat krusial untuk menemukan kebocoran data (*Data Exfiltration*).
3. **Amazon GuardDuty (The IDS):** Layanan deteksi ancaman awan bawaan AWS. Menggunakan *Machine Learning* untuk menganalisis log *CloudTrail*, *VPC*, dan *DNS* untuk memberikan alarm saat mendeteksi indikator kompromi (seperti IP penambang kripto atau panggilan API aneh dari lokasi geografis yang tidak biasa).

## 2.2 Berburu Anomali IAM via Amazon Athena (SQL)

*CloudTrail* menyimpan catatannya dalam format JSON kompresi yang sangat masif di *Amazon S3*. Anda tidak bisa membacanya menggunakan *Notepad*. Analis DFIR menggunakan **Amazon Athena**, sebuah layanan yang memungkinkan Anda melakukan kueri SQL langsung ke berkas JSON di *Bucket S3* tersebut.

```
# Triage SQL: Menemukan Siapa yang Memberikan Hak Akses Penuh (Admin) ke Akun Pengguna SELECT eventTime, userIdentity.arn AS Pelaku, eventName AS Aksi, requestParameters FROM cloudtrail_logs WHERE eventName = 'AttachUserPolicy' AND requestParameters LIKE '%AdministratorAccess%' ORDER BY eventTime DESC LIMIT 10;
```

Jika kueri ini mengembalikan hasil pada jam 3 pagi yang dilakukan oleh akun layanan biasa, Anda baru saja menemukan aktivitas **Privilege Escalation** (Peretas menaikkan jabatannya menjadi Admin AWS). Langkah respons instan adalah segera menonaktifkan akun pelaku tersebut dan merotasi kuncinya (*Key Rotation*).

## 2.3 SSRF Attacks & Pencurian Kredensial IMDS

Serangan awan paling legendaris (seperti Peretasan Capital One) menggunakan kerentanan **SSRF (Server-Side Request Forgery)**. Peretas menyerang *web server* (EC2) dari internet, memaksanya untuk melakukan kueri ke alamat IP sihir internal AWS (`169.254.169.254`).

Alamat IP ini adalah **Instance Metadata Service (IMDS)**. Jika diakses, ia akan memuntahkan kredensial Kunci Rahasia AWS (*Temporary IAM Credentials*) milik peran (*Role*) EC2 tersebut. Peretas kemudian mengekstrak kunci tersebut ke laptop mereka sendiri (di Rusia/Tiongkok) dan masuk ke akun AWS Anda menggunakan kunci asli Anda.

**Forensik & Mitigasi:**
Di CloudTrail, Anda akan melihat indikator *Console Login* atau pemanggilan API AWS dengan Kredensial EC2, tetapi alamat IP Asalnya (*Source IP*) bukan berasal dari IP internal VPC, melainkan IP Internet eksternal. Untuk mencegah hal ini, arsitek awan harus mematikan IMDSv1 lama dan mewajibkan penggunaan **IMDSv2**, yang menggunakan perlindungan kriptografi *Token* sehingga kebal terhadap serangan SSRF aplikasi web.

## 2.4 Investigasi Ransomware di Amazon S3

*Ransomware* di infrastruktur awan tidak menargetkan sistem operasi (*Drive C:*), mereka menargetkan *Object Storage* (Amazon S3) yang menyimpan petabyte data cadangan (*Backup*) perusahaan.

Taktik *Ransomware S3* modern tidak repot-repot mengunduh, mengenkripsi, lalu mengunggah kembali (terlalu lambat). Peretas hanya menggunakan panggilan API KMS (*Key Management Service*) untuk mengubah kunci enkripsi keranjang (*Bucket*) Anda menjadi kunci KMS milik peretas sendiri. Atau, mereka memanggil API penghapusan massal.

**Perburuan DFIR:**
Analis akan memeriksa *CloudTrail Data Events* (bukan sekadar *Management Events*). Kueri Athena difokuskan pada lonjakan (*Spike*) yang masif pada pemanggilan API `DeleteObject`, `PutObject`, atau `PutBucketEncryption` dalam kurun waktu 15 menit.
Pemulihan awan jauh lebih cepat dari *On-Premise*: Jika perusahaan menghidupkan fitur *S3 Object Versioning* dan *MFA Delete*, insinyur awan dapat membatalkan (*Rollback*) kerusakan *ransomware* tersebut dengan satu kueri *Python boto3*, memulihkan miliaran berkas ke versi detik sebelum penghapusan terjadi.

## 2.5 Snapshot Forensics: Mengakuisisi Volume EC2

Jika *Server* Windows (EC2) terkena infeksi *Web Shell* dan Anda membutuhkan investigasi *hard disk* penuh, jangan pernah mematikan (*Stop/Terminate*) *server* tersebut, karena memori RAM dan data sementara (*Ephemeral Drives*) akan hangus.

Metode **Cloud-Native Disk Acquisition** adalah sebagai berikut:

1. Pisahkan jaringan mesin terinfeksi: Ubah *Security Group* EC2 tersebut sehingga seluruh aturan lalu lintas masuk dan keluar (*Inbound/Outbound*) terhapus, mengkarantinanya sepenuhnya dari internet dan *subnet* internal.
2. Gunakan API AWS untuk mengambil **EBS Snapshot** (Citra Disk) dari *Volume Hard Disk* mesin terinfeksi saat mesin masih menyala (*Live Snapshot*).
3. Bagikan *Snapshot* tersebut ke akun AWS Forensik (Akun khusus terisolasi).
4. Nyalakan *Virtual Machine Forensik* (SIFT Workstation Linux) di akun Forensik, pasangkan (*Mount*) *Snapshot* tadi sebagai *disk* sekunder Read-Only, dan lakukan analisis *Timeline* (Plaso/Log2Timeline) tanpa menyentuh *server* produksi yang terinfeksi.

## 2.6 Mengekstrak Jejak Jaringan (VPC Flow Logs)

Untuk membuktikan kebocoran data (*Data Exfiltration*), Anda tidak memiliki PCAP penuh. Anda mengandalkan **VPC Flow Logs**. Kueri ini akan mencari IP internal (Misal *Server Database* `10.0.1.55`) yang tiba-tiba mengirimkan data berukuran raksasa ke IP Internet Rusia (Indikator pencurian DB).

```
# AWS Athena: Mendeteksi Volume Transfer Data (Exfiltration) ke Luar Jaringan VPC SELECT srcaddr AS Sumber_IP_Internal, dstaddr AS Tujuan_IP_Eksternal, dstport AS Port, SUM(bytes) / 1048576 AS Total_Megabytes_Dikirim FROM vpc_flow_logs WHERE action = 'ACCEPT' AND dstaddr NOT LIKE '10.%' -- Bukan jaringan internal GROUP BY srcaddr, dstaddr, dstport HAVING SUM(bytes) > 500000000 -- Menampilkan yang mengirim > 500 MB ORDER BY Total_Megabytes_Dikirim DESC;
```

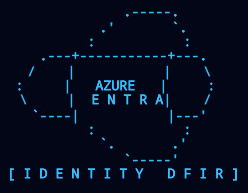
Laporan keluaran kueri ini memberikan bukti matematis besaran kerugian (berapa Megabyte/Gigabyte data yang berhasil dipindahkan oleh musuh) untuk diserahkan ke pelaporan manajemen krisis (*C-Level Incident Report*).

## 2.7 Analisis Keterlibatan Akun Pihak Ketiga (Cross-Account)

Arsitektur AWS modern tidak menggunakan satu akun (seperti portal *login* tunggal). Perusahaan menggunakan **AWS Organizations** dengan ratusan akun AWS (*Dev, QA, Prod, Billing*). Untuk beralih antar akun, layanan menggunakan *Cross-Account Roles*.

Peretas menggunakan fitur ini untuk menghindari deteksi. Mereka mungkin meretas masuk ke akun *Dev* (yang penjagaannya longgar), lalu menemukan izin IAM (`sts:AssumeRole`) yang memungkinkan mereka melompat melintasi batas ke akun *Prod* (Produksi Keuangan).

Dalam investigasi CTI, analis harus menyusun teka-teki (*Stitching*) dengan menggabungkan log *CloudTrail* dari Akun Dev dan Akun Prod secara bersamaan. Di Akun Prod, Anda akan melihat panggilan API dilakukan oleh Aktor `arn:aws:sts::ProdAccount:assumed-role/CrossRole/Session1`. Identitas aslinya disembunyikan. Anda harus membawa ID Sesi (*Session ID*) tersebut, mencari log di Akun Dev pada cap waktu (*timestamp*) yang sama, untuk akhirnya mengungkap IP asli peretas tersebut.




# BAGIAN III

Azure DFIR & Entra ID
(Identity and Access Forensics)

# BAB 3: MICROSOFT AZURE DFIR

## 3.1 Lanskap Log Azure (Activity Log, NSG Flow)

Beralih ke ekosistem awan Microsoft Azure, terminologi berubah namun filosofi investasigasinya sama. Platform awan tidak mendeteksi *malware* di memori, mereka mendeteksi penyalahgunaan infrastruktur.

Tiga sumber bukti utama di ekosistem Microsoft Azure adalah:

1. **Azure Activity Logs:** (Sama dengan AWS CloudTrail). Mencatat peristiwa sarana kendali (*Control Plane*) seperti pembuatan Virtual Machine, modifikasi *Network Security Group (NSG)*, dan perubahan pengaturan kunci *Azure Key Vault*.
2. **Azure NSG Flow Logs:** (Sama dengan VPC Flow Logs). Metadata catatan paket IP (Masuk dan Keluar) yang diizinkan atau ditolak oleh aturan *Firewall* NSG awan Anda.
3. **Microsoft Entra ID Logs (Dulu Azure AD):** Ini adalah jantung keamanan korporat (*Enterprise*). Berisi log Autentikasi (Siapa yang masuk ke Office 365, lokasi negara masuknya) dan log Audit (Pembuatan pengguna baru, perubahan izin administrator global). Di Azure, Identitas adalah perimeter sejati (*Identity is the new perimeter*).

## 3.2 Triage Log Entra ID (Otentikasi Kondisional)

Lebih dari 90% peretasan korporat awan (termasuk kompromi email bisnis / BEC) berawal dari kebocoran akun *Entra ID (Azure AD)*. Saat merespons insiden (IR), fokus utama analis *Threat Hunting* adalah mengaudit halaman **Sign-in Logs**.

Analis mencari anomali seperti:

- **Impossible Travel (Perjalanan Mustahil):** Pengguna *Budi* masuk dari Jakarta (Indonesia) pada jam 09:00, lalu berhasil masuk lagi dari Lagos (Nigeria) pada jam 09:10. Mustahil melakukan perjalanan pesawat dalam 10 menit.
- **MFA Fatigue (Kelelahan MFA):** Musuh yang memiliki kata sandi karyawan akan membombardir ponsel karyawan tersebut dengan ratusan persetujuan *Push Notification* otentikator pada jam 2 pagi. Berharap karyawan yang mengantuk tersebut tanpa sadar menekan "Setuju" agar ponselnya berhenti berbunyi. Ini tercatat jelas sebagai 100 log `MFA Denied` diikuti 1 `MFA Approved` di Entra ID.

## 3.3 Serangan Illicit Consent Grant (OAuth Phishing)

Musuh tahu bahwa perusahaan Anda sudah menggunakan *MFA (Otentikasi Multi-Faktor)*, sehingga mencuri kata sandi tidak lagi cukup. Mereka menggunakan taktik **Illicit Consent Grant (Phishing OAuth)**.

Anda menerima email tentang pembaruan aplikasi *HR Payroll*, saat di-klik, halaman resmi Microsoft muncul menanyakan: *"Aplikasi 'HR Payroll 2026' meminta izin untuk **Membaca Email Anda**, **Mengirim Email Atas Nama Anda**, dan **Membaca Semua File SharePoint**. [Accept]"*.

Jika pengguna menekan *Accept*, penyerang mendapatkan *OAuth Token* (Tiket emas). Mereka tidak membutuhkan kata sandi Anda dan mereka menembus (*bypass*) MFA Anda sepenuhnya karena akses mereka difasilitasi resmi oleh delegasi *Microsoft Graph API*.

**Forensik:** Analis CTI akan melakukan kueri pada tabel *Audit Logs Entra ID* mencari *Event Action* bernama `Consent to application`. Analis kemudian mencabut izin token aplikasi OAuth palsu (*Enterprise Application*) tersebut untuk menghentikan akses peretas dari pintu belakang (*Backdoor*).

## 3.4 Golden SAML & Perburuan Federasi Kredensial

Serangan paling menakutkan dalam sejarah *Cloud* (seperti Insiden *SolarWinds* yang meretas pemerintah AS) menggunakan teknik **Golden SAML**.

Perusahaan sering menggunakan Identitas Gabungan (*Federated Identity*) di mana peladen fisik (*Active Directory ADFS*) di pusat data perusahaan mengeluarkan tiket (SAML *Token*) yang ditandatangani secara kriptografis untuk masuk tanpa kata sandi ke dalam *Cloud Azure/Microsoft 365*.

Dalam Golden SAML, peretas meretas *server* fisik ADFS, mencuri sertifikat rahasia penandatangan tiket tersebut. Kini, peretas dapat memalsukan tiket SAML (Mencetak paspor palsu yang terlihat 100% asli) atas nama siapa saja (termasuk Direktur Utama). Saat mereka masuk ke awan Azure, *Cloud* tidak mendeteksi *Login* paksa (*Brute Force*) karena tiketnya ditandatangani oleh sertifikat kripto resmi perusahaan Anda.

Menginvestigasi ini sangat sulit karena jejak masuk (*Sign-ins*) terlihat sah. Tim IR harus berkorelasi lintas lanskap: Mencari peristiwa pengunduhan sertifikat rahasia (*Export Key*) di log peladen *Windows On-Premise*.

## 3.5 Berburu Ancaman Menggunakan KQL (Microsoft Sentinel)

Seluruh log Azure, Office 365, dan Entra ID mengalir secara otomatis ke SIEM Cloud-Native Microsoft, yakni **Microsoft Sentinel**. Analis DFIR Sentinel menggunakan **KQL (Kusto Query Language)**, sebuah bahasa pencarian data raksasa (*Big Data*) yang jauh lebih cepat daripada SQL.

```
// KQL Triage: Mendeteksi Admin Entra ID yang Masuk (Login) Tanpa MFA  // dari Lokasi Geografis Eksternal (Luar Negeri). Indikator Kuat Kompromi. SigninLogs | where AuthenticationRequirement == "singleFactorAuthentication" | where ResultType == "0" // 0 = Login Sukses | where LocationDetails.countryOrRegion != "ID" // Bukan dari Indonesia // Menggabungkan tabel otentikasi dengan tabel hak istimewa (Roles) | join kind=inner ( AuditLogs | where OperationName == "Add member to role" | where TargetResources[0].displayName == "Global Administrator" ) on $left.UserPrincipalName == $right.InitiatedBy.user.userPrincipalName | project TimeGenerated, UserPrincipalName, Location, IPAddress, AppDisplayName
```

Kemahiran seorang Insinyur *Cloud Security* (Azure) diukur dari kefasihannya merajut skema tabel-tabel data KQL secara waktu nyata (*Real-time Hunting*).

## 3.6 Mengisolasi Azure Virtual Machines (NSG)

Jika Sentinel membunyikan alarm bahwa *Virtual Machine (VM) Azure* Anda terinfeksi *malware* yang sedang menghubungi pusat perintah di Korea Utara, Anda harus mengisolasi VM tersebut secara forensik, persis seperti mencabut kabel LAN di lingkungan fisik.

Langkah penanggulangannya (*Containment*) adalah menggunakan **Network Security Groups (NSG)** Azure:

1. Jangan pernah menghapus atau mematikan (*Deallocate*) VM. Jika Anda melakukannya, RAM akan terhapus dan alamat IP Publik akan dilepas, menghancurkan barang bukti (*Evidence Spoliation*).
2. Buat dan terapkan aturan NSG Prioritas Tertinggi (Prioritas 100): `Deny All Inbound` (Tolak Semua Masuk) dan `Deny All Outbound` (Tolak Semua Keluar).
3. Terapkan satu-satunya pengecualian (Prioritas 99): Izinkan akses masuk (SSH/RDP) **HANYA** dari alamat IP Statis Kantor Pusat SOC / IP Laptop Tim Forensik Anda.

Sekarang VM tersebut lumpuh, infeksi tidak bisa menyebar (Lateral), *malware* terputus dari *Server C2*, tetapi Tim Forensik tetap bisa masuk untuk mengumpulkan *Memory Dump* (Menggunakan utilitas *DumpIt* / *LiME*).




# BAGIAN IV

Kubernetes (K8s) &
Container Forensics

# BAB 4: KUBERNETES FORENSICS

## 4.1 Permukaan Serangan Klaster K8s

Platform komputasi korporat paling populer hari ini tidak lagi berupa VM (*Virtual Machine*), melainkan orkestrasi kontainer menggunakan **Kubernetes (K8s)**. Arsitektur K8s (yang terdiri dari *Pods, Nodes, Kubelet*, dan *API Server*) memiliki permukaan serangan yang unik.

Di K8s, satu peladen *Host/Node* (VM fisik/AWS EC2) menampung puluhan kontainer aplikasi (*Pods*). Kontainer dirancang agar aman, berjalan di ruang (*Namespace*) terisolasi (*Sandboxed*) menggunakan teknologi isolasi kernel Linux (*cgroups* dan *namespaces*).

Namun, jika peretas berhasil mengeksploitasi aplikasi web (*Apache/Nginx*) di dalam kontainer yang dikonfigurasi secara ceroboh (*Misconfigured*), tujuan mereka adalah melakukan **Container Escape (Pelolosan Kontainer)**. Mereka ingin menghancurkan dinding pasir (*sandbox*), keluar dari kontainer kecil mereka, dan mengambil alih kendali *Root* peladen (Node) *Host* utama di bawahnya. Begitu *Node* jatuh, seluruh klaster perusahaan hancur.

## 4.2 Triage K8s API Audit Logs

Serupa dengan AWS *CloudTrail*, sarana kendali Kubernetes dioperasikan secara penuh melalui pemanggilan *API Server*. Analis Forensik harus menganalisis **Kubernetes Audit Logs**.

Analis mencari perintah (*Commands*) mencurigakan yang dieksekusi oleh *Service Accounts* (Akun bot/mesin) atau administrator liar, misalnya mengeksekusi kerang *Terminal (Shell)* langsung ke dalam *Pod* yang sedang berjalan (sama berbahayanya dengan membuka RDP).

```
# Triage Kibana/Elastic: Mencari Siapa yang Mencoba Masuk (Exec Shell) ke Dalam Pod Basis Data
kubernetes.audit.objectRef.resource: "pods" AND kubernetes.audit.objectRef.subresource: "exec" AND
kubernetes.audit.verb: "create" AND kubernetes.audit.requestURI: "*sh*" # Indikator Kompromi (IoC):
Pembuatan Pod dengan Akses Privileged (Admin)
kubernetes.audit.requestObject.spec.containers.securityContext.privileged: true
```

Deteksi penambahan kontainer dengan hak istimewa (`privileged: true`) adalah indikator serangan tingkat dewa (*Critical Priority*). Peretas akan menggunakan kontainer berprivilese ini untuk memformat *hard disk Node*.

## 4.3 Anatomi Container Escape

Tim DFIR menganalisis celah kompromi yang paling umum di K8s: **Penyalahgunaan HostPath Mounts**. Sebuah aplikasi (*Pod*) terkadang perlu membaca arsip (*file*) sistem *host*. Pengembang menempelkan (*Mount*) direktori akar (*Root Directory* `/`) dari *Server Host* (VM Linux) ke dalam direktori kontainer (misalnya ke dalam folder `/mnt/host_root/`).

Jika peretas meretas *Pod web* tersebut, mereka sekarang memiliki akses baca/tulis ke arsip *Host Server* di folder `/mnt/host_root/`. Peretas lalu bernavigasi ke dalam folder tersebut (yang sebenarnya adalah otak sistem operasi induknya), mencari fail penjadwal tugas (`cron.d`) atau Kunci SSH, lalu menambahkan perintah *Backdoor* milik mereka sendiri.

Dalam 1 menit kemudian, sistem operasi peladen *Host* utama secara buta mengeksekusi *Backdoor* peretas tersebut sebagai administrator *Root*, mengira perintah itu berasal dari dirinya sendiri. *Container Escape* telah sukses dilakukan tanpa perlu alat peretasan (*Exploit*) *Zero-Day* apa pun; hanya melalui kesalahan konfigurasi volume (*Mounting*).

## 4.4 Penyalahgunaan Docker Socket (docker.sock)

Terkadang, aplikasi *CI/CD (Jenkins/GitLab)* yang berjalan di dalam kontainer perlu kemampuan untuk membuat atau menghapus kontainer Docker lainnya (*Docker in Docker*). Untuk mencapai hal ini, pengembang memaparkan fail jaringan `/var/run/docker.sock` langsung ke dalam kontainer tersebut.

Fail *docker.sock* ini adalah antarmuka sakral (API UNIX) yang dapat memerintah seluruh mesin Docker lokal tanpa kata sandi apa pun.

Jika penyerang menyusup ke kontainer yang memiliki `docker.sock` terekspos, ini sama berbahayanya dengan memberikan tombol nuklir. Forensik akan menunjukkan peretas memanggil API *Socket* ini (menggunakan CURL internal) untuk memerintahkan Docker peladen induk agar membuatkan kontainer *Ubuntu* baru yang memuat *malware*, dan secara instan memberikan akses administrator (*Root Privilege*) kepadanya. Penyerang kini bebas menjajah (*Lateral Movement*) ke seluruh mesin di klaster perusahaan.

## 4.5 Teknik Akuisisi Memori (Sidecar Pattern)

Bagaimana Anda mengambil tangkapan memori RAM (*Memory Dump*) dari sebuah kontainer *Pod* yang terinfeksi? Anda tidak dapat menekan tombol *Hibernate* (Jeda) fisik.

Arsitektur K8s yang mutakhir (seperti *Falco* atau agen CWPP) menggunakan konsep **Forensic Sidecar (Kontainer Pendamping)**.

Dalam *Kubernetes*, beberapa kontainer yang ditempatkan di dalam satu *Pod* yang sama dapat saling berbagi ruang nama (*Namespace*) memori dan jaringan. Jika sistem deteksi (SIEM) mendeteksi *malware* di kontainer `Aplikasi_Web`, skrip otomatis SOAR akan secara dinamis menyuntikkan kontainer baru (`Forensic_Sidecar`) ke dalam *Pod* yang sama.

Kontainer *Sidecar* ini telah dilengkapi alat keamanan berat (seperti TCPDump, LiME, Volatility). Ia bertugas merekam lalu lintas jaringan si *malware* dan mencetak memori RAM-nya, menyimpannya di Amazon S3, lalu kontainer *Sidecar* ini membunuh dirinya sendiri (hilang) dalam 5 menit. Sebuah akuisisi forensik otomatis yang bersih (*Surgical*) dan awan-asli (*Cloud-Native*).

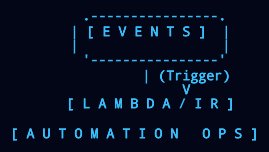
## 4.6 Membekukan Kontainer (Cordon & Pause)

Langkah karantina (*Containment*) di Kubernetes memiliki perbendaharaan kata yang berbeda dari lingkungan jaringan fisik. Jika *Pod* terinfeksi, Anda tidak boleh menekan tombol Hapus (`kubectl delete pod`).

Jika Anda menghapusnya, *Deployment Controller* K8s akan menganggap *Pod* tersebut *Crash* dan **akan secara otomatis membuat Pod baru (Replikasi)** untuk menggantikannya. Peretas akan senang karena *malware* mereka hidup kembali secara otomatis, sementara Anda telah membuang memori *Pod* yang pertama yang berisi bukti kunci.

**Langkah Forensik yang Benar:**

1. Hapus label (*Labels*) dari *Pod* terinfeksi. Ini akan mengeluarkannya dari perutean *Load Balancer* (Karantina Jaringan), tetapi *Pod* masih tetap hidup untuk dianalisis.
2. Jalankan `kubectl cordon [NAMA-NODE]` pada *Server Host*-nya agar Kubernetes berhenti menjadwalkan *Pod* baru ke *Server* yang terkompromi tersebut.
3. Setelah bukti memori disalin (*Imaging*), eksekusi proses penghentian kontainer dari dalam Docker *Host* dengan aman.




# BAGIAN V

Serverless & Automated
Incident Response (SOAR)

# BAB 5: SERVERLESS & AUTO-IR

## 5.1 Lanskap Forensik Nirserver (AWS Lambda)

Evolusi komputasi puncak saat ini adalah **Serverless (Nirserver)**, seperti AWS Lambda atau Azure Functions. Pengembang hanya menempelkan potongan *script Python* atau *NodeJS* di layar AWS. Jika pengguna mengunjungi situs web, AWS akan membuat lingkungan komputasi dalam hitungan milidetik, mengeksekusi skrip, lalu menghancurkan komputasi tersebut detik itu juga (*Micro-VM Firecracker*).

**Bencana Forensik:** Tidak ada sistem operasi (*OS*) untuk Anda pancing memori RAM-nya (*No VM, No Memory Dump*). Anda tidak dapat memasang Antivirus (CWPP) di dalamnya. Waktu hidup aplikasi (*Lifespan*) hanya sekian ratus milidetik.

Keamanan *Serverless* sangat bergantung murni pada manajemen IAM (*Identities*) dan pemantauan pergerakan API. Jika skrip fungsi *Lambda* Anda rentan terhadap *Command Injection* (karena pengembang gagal menyaring *input* pengguna), peretas tidak akan meretas *server*-nya (karena *server*-nya akan mati dalam sedetik), melainkan akan menyuruh fungsi tersebut untuk memanggil *Database* pusat (Mengeksfiltrasi Data) dengan wewenang sah (*IAM Role*) milik fungsi tersebut.

## 5.2 Mendeteksi Backdoor pada Fungsi Lambda

Tim DFIR menangani penyusupan *Serverless* dengan audit lapisan kendali (*Control Plane Logging*). Jika musuh mencuri akun *Developer* AWS yang kompromi, mereka dapat diam-diam menambahkan skrip nakal ke dalam fungsi *AWS Lambda* layanan pembayaran Anda (*Backdooring the code*).

Skrip tersebut direkayasa agar menyalin 10% dari setiap data kartu kredit pelanggan, dan mengirimkannya ke API eksternal peretas di Tiongkok secara senyap setiap fungsi tersebut dipanggil.

Untuk mendeteksi ini, *Security Engineer* menggunakan **CloudTrail Data Events** dan mencocokkan *Hash* integritas (*SHA-256 Checksum*) dari kode ZIP *Lambda*. Jika kode *Lambda* diperbarui pada waktu yang tidak wajar (pukul 3 pagi di luar jendela rilis *CI/CD Pipeline GitLab* yang resmi), itu adalah indikasi 100% *Backdoor*.

Fungsi forensik AWS Lambda juga membutuhkan pemantauan *VPC Flow Logs* untuk mendeteksi anomali volume pertukaran jaringan keluaran (*Outbound Network Traffic*) ke IP yang tidak dikenal (*Unsanctioned APIs*).

## 5.3 Automated Containment Architecture

Kecepatan serangan awan tidak dapat dihentikan oleh manusia yang sedang tidur, atau Analis L1 yang menghabiskan waktu 10 menit hanya untuk membaca peringatan (*Alert*) SIEM. Respons di dalam awan (*Cloud IR*) harus dimediasi oleh kode (**Security Orchestration, Automation, and Response / SOAR**).




Arsitektur penanggulangan (*Containment*) otomatis yang elegan beroperasi seperti saraf refleks: Mesin *Threat Intel* (GuardDuty/SIEM) mendeteksi anomali kritikal, mengirimkan paket muatan insiden (*Event JSON*) melalui penghala pesan awan (*AWS EventBridge/Azure Event Grid*). Penghala tersebut seketika (dalam 2 detik) membangunkan bot respons *AWS Lambda* (Python) yang akan mengeksekusi API pertahanan (*Playbook Automations*).

## 5.4 Playbook: Otomasi Pencabutan Kredensial

Skenario Terburuk: Detektor *GuardDuty* Anda menemukan bahwa *IAM Token* milik aplikasi produksi telah dicuri dan sedang digunakan dari negara Tiongkok. Anda tidak punya waktu untuk menghubungi administrator IAM. Fungsi *Lambda (IR Bot)* Anda akan membaca peringatan insiden (JSON) dan secara otomatis mencabut sesi token aktif API penyerang (`RevokeSessions`).

```
# Python AWS Boto3: Mencabut Sesi Token Kredensial Sementara Penyerang (STS) import boto3 import
datetime def lambda_handler(event, context): iam_client = boto3.client('iam') kompromi_role =
event['detail']['resource']['roleName'] # Memaksa agar SEMUA token yang diterbitkan sebelum detik ini
tidak lagi valid response = iam_client.put_role_policy( RoleName=kompromi_role,
PolicyName='DenyAllPreviousTokens', PolicyDocument={ "Version": "2012-10-17", "Statement": [{ "Effect":
"Deny", "Action": "*", "Resource": "*", "Condition": {"DateLessThan": {"aws:TokenIssueTime":
str(datetime.datetime.utcnow())}} }] } ) return "Pencabutan Token Sukses. Penyerang Terlempar dari
Cloud."
```

## 5.5 Playbook: Isolasi EC2 Terinfeksi

Skenario Kedua: EDR (CrowdStrike) mendeteksi instalasi *Ransomware* di dalam peladen virtual *Windows EC2* (IP 10.0.1.55). Bot respons *Lambda* Anda akan segera melepaskan (*Detach*) EC2 tersebut dari jaringan perusahaan dan menempelkannya ke *Forensic Security Group*, memblokir seluruh pergerakan Lateral.

```
# Python AWS Boto3: Mengkarantina EC2 Terinfeksi dari Jaringan import boto3 def
isolate_instance(instance_id, quarantine_sg_id): ec2_resource = boto3.resource('ec2') instance =
ec2_resource.Instance(instance_id) # Menyimpan Grup Keamanan lama (untuk log forensik) old_sg =
[sg['GroupId'] for sg in instance.security_groups] print(f"Grup Keamanan lama: {old_sg}") # Membuang
Grup Keamanan lama, menempelkan Grup Karantina Khusus  # (Inbound & Outbound DENY ALL, kecuali SSH dari
IP Forensik) instance.modify_attribute(Groups=[quarantine_sg_id]) return f"Instans {instance_id} sukses
dikarantina dari internet."
```

Hanya dengan skrip kecil berukuran 200 *bytes* ini, seluruh ancaman penyebaran *malware* terhenti secara instan dalam skala yang mustahil dilakukan manusia.

## 5.6 Remediasi Postur Keamanan (Shift-Left)

Fase akhir dari respons insiden modern adalah menutup celah awal (*Root Cause*) secara terprogram. Di dunia awan, jika seorang *Developer* secara ceroboh mengubah pengaturan (ACL) sebuah *Bucket S3* perusahaan menjadi "Public Read/Terbuka Ke Internet", SOAR/CNAPP harus memperbaiki konfigurasi kaku tersebut (*Self-Healing Infrastructure*).

Arsitektur keamanan modern tidak menunggu peretas mengunduh fail dari keranjang tersebut, atau menunggu tim auditor keamanan memberikan laporan audit kepatuhan (*Compliance Report*) di akhir bulan. Sebuah Fungsi *Lambda* akan menyala dalam rentang 3 detik setelah *CloudTrail* mencatat API perubahan konfigurasi, mengevaluasi apakah perubahan tersebut sesuai dengan Kebijakan (*Policy*) perusahaan, dan jika melanggar, skrip otomatis tersebut memanggil API `PutBucketAcl` untuk mengembalikan keranjang *S3* tersebut ke status `Private` (Tertutup) detik itu juga.

Inilah yang disebut paradigma **Shift-Left Security** dan Otomatisasi Remediasi (*Automated Remediation*). Manusia merancang aturan, mesin yang menegakkannya.

## 5.7 Mempertahankan Chain of Custody (Rantai Bukti)

Kelemahan terbesar *Cloud DFIR* di pengadilan adalah keabsahan bukti elektronik (*Chain of Custody*). Jika IP musuh Anda buktikan dari log *S3 Bucket CloudTrail* Anda, musuh (Aktor internal) dapat berdalih: *"Log tersebut bisa saja Anda (Tim Keamanan) palsukan secara manual di dalam Bucket S3 tersebut untuk memberatkan saya."*

Insinyur Awan (*Cloud Security Architect*) mencegah penyangkalan ini dengan menerapkan kontrol kriptografi. Semua *Bucket S3* forensik (Tempat mendaratnya log CloudTrail dan Azure Activity Logs) harus secara absolut mengaktifkan fitur **S3 Object Lock (Mode Kepatuhan/Compliance Mode)**.

Konfigurasi *WORM (Write Once, Read Many)* ini dikunci di tingkat infrastruktur perangkat keras *AWS/Azure*. Setelah fail log ditulis ke dalam keranjang tersebut, **Tidak ada satu entitas pun di bumi ini** (Bahkan Akun *Root AWS* Anda sendiri, atau tim Dukungan *AWS*) yang dapat menghapus, memodifikasi, atau menyamarkan sebaris huruf dalam log tersebut sebelum masa retensi (Misal 5 Tahun) berakhir. Bukti log Anda kini memiliki integritas yang terjamin dan kebal hukum.

## 5.8 Integrasi Forensik Multi-Cloud

Perusahaan berskala global (*Enterprise*) jarang sekali bertahan pada komitmen tunggal (*Single Cloud*). Mereka secara strategis akan menjalankan basis data komputasi berat di **AWS**, menyelaraskan identitas dan sistem surel kolaboratif (Office 365) menggunakan **Microsoft Azure**, serta menjalankan kerangka kerja kecerdasan buatan (*BigQuery / AI-Ops*) di **Google Cloud (GCP)**.

Ancaman utama dalam komputasi Multi-Awan (*Multi-Cloud*) adalah fragmentasi investigasi (*Siloed DFIR*). Jika Kredensial *SSO/SAML* di Azure dicuri, musuh akan menggunakannya untuk masuk secara terselubung ke *AWS Console* untuk menyalakan *Server* *Cryptomining* tersembunyi.

Tim *Incident Response (IR)* harus menghubungkan titik (*Connecting the dots*) pergerakan musuh lintas batas vendor awan. Untuk mengatasi fragmentasi data, Arsitek Keamanan membangun **Data Lake Sentral (Seperti Elastic Security atau Splunk Cloud)** yang secara asinkron (*Asynchronous*) menelan log mentah dari *AWS CloudTrail*, *Azure Activity*, dan *GCP Audit Logs* menggunakan saluran (*Pipelines*) Logstash/Kafka. Data dinormalisasi ke dalam *Elastic Common Schema (ECS)* sehingga analis dapat mengendus perburuan ancaman (*Threat Hunting*) menggunakan satu kueri *KQL/SPL* global yang menjahit identitas penyerang tunggal lintas dimensi Awan.

## 5.9 Evaluasi Kesiapan Tim Respons Insiden (Tabletop Exercise)

Memiliki *Playbook* Python *Serverless* yang super otomatis dan dasbor SIEM KQL yang mewah tidak menjamin kemenangan di malam insiden (*Breach Day*). Kesiapan operasional (*Operational Readiness*) dari tim *Security Operations Center (SOC)* dan Tim *DevOps* harus diuji secara komprehensif tanpa simulasi skrip murni.

Metodologi pengujian kesiapan krisis eksekutif disebut **Cloud Tabletop Exercise (TTX)** (Latihan Meja Bundar). Fasilitator keamanan siber (CISO atau Konsultan Eksternal) akan melemparkan skenario horor (*Doomsday Scenario*) tanpa peringatan: *"Peretas telah menyandera (Ransomware) S3 Bucket utama kita. Mereka juga telah mengubah kunci enkripsi KMS, dan membocorkan sampel data nasabah VIP kita di Twitter (X). Direktur PR (Public Relations) menuntut penjelasan dalam 10 menit. Apa langkah taktis pertama tim SOC?"*

Latihan TTX (Meja Bundar) mengekspos patahan (*Friction*) krusial yang bersifat non-teknis, seperti kegagalan komunikasi internal (*Miscommunication*), tidak ada kepastian siapa yang memegang *AWS Root Account MFA (Master Key)* dalam situasi krisis, dan ketidakmampuan berkoordinasi dengan dewan penasihat hukum (*Legal Council*). Kematangan DFIR adalah sinergi antara mesin kueri log awan yang akurat dan protokol komunikasi krisis manusia (*Human Crisis Management*) yang tenang dan terkalkulasi.

## 5.10 Visibilitas Ancaman Tingkat Lanjut: Memory Scraping

Meskipun investigasi awan difokuskan pada manipulasi identitas (IAM) API *Control Plane*, musuh tingkat APT (*Advanced Persistent Threat*) yang sangat canggih terkadang membutuhkan interaksi memori level terendah untuk mengekstrak kata sandi rahasia (*Vault Secrets*) langsung dari RAM OS Linux EC2.

Serangan tingkat OS seperti modul *Kernel Rootkits* (Contoh: BPFFilter/Khide) akan bersembunyi dari perintah `ps` atau `top` di Linux, dan tidak menghasilkan *Event Log* jaringan awan sama sekali karena operasi ini sepenuhnya terselubung (*Stealth*).

Untuk menembus kebutaan visibilitas (*Blind Spot*) pada komputasi IaaS (*Infrastructure as a Service*), *Threat Hunter* mutlak harus menyebarkan agen CWPP (*Cloud Workload Protection Platform*). Agen ini menggunakan kemampuan pemantauan subsistem Kernel Linux (*eBPF - Extended Berkeley Packet Filter*) terbaru untuk melihat ke dalam tumpukan memori proses yang sedang aktif tanpa memberikan penalti (*Overhead*) berat terhadap beban *Server* VM. Jika *Rootkit* menyusup ke *Memory Space*, CWPP akan langsung mendeteksi pola tanda tangan tak normal dan mencabut jaringan VM tersebut melalui integrasi API Awan.

## 5.11 Manajemen Forensik Hukum dan Privasi (GDPR)

Lanskap operasi penyedia layanan Awan Publik (*Public Cloud*) melintasi batasan kedaulatan negara (*Data Sovereignty*). AWS dan Azure menyimpan log telemetri dan data cadangan investigasi forensik Anda di pusat data geospasial (*Regions/Availability Zones*) yang membentang dari Amerika Serikat hingga Frankfurt dan Tokyo.

Jika investigasi internal DFIR pada klaster Awan Eropa secara otomatis menyedot *(Dump)* data cadangan memori (RAM) dari *Virtual Machine* yang tak sengaja memuat nama identitas atau alamat surel pelanggan Uni Eropa (PII), lalu memindahkan (*Transfer*) arsip *Memory Dump* PCAP raksasa tersebut ke *Bucket S3* forensik yang terletak di Data Center AWS Virginia (Amerika Serikat), perusahaan Anda baru saja melanggar kepatuhan undang-undang **GDPR (General Data Protection Regulation)** Pasal transfer data lintas-batas (*Cross-Border Transfer Violation*).

Arsitektur keamanan *Cloud DFIR* tidak boleh ceroboh dalam sentralisasi lalu lintas. Penyimpanan log sentral dan proses akuisisi bukti forensik digital (*Data Residency*) **HARUS** dikonfigurasi terikat-wilayah (*Geo-Fenced*) agar tetap mematuhi hukum data lokal tanpa menghalangi kelancaran perburuan ancaman secara global.

## 5.12 Post-Incident Review (PIR) dan Refactoring

Setiap operasi DFIR di lingkungan komputasi awan yang berakhir dengan *karantina mesin otomatis* akan bermuara pada tahap penyusunan **Laporan Tinjauan Pasca Insiden (Post-Incident Review / PIR)**. Laporan ini bukan sekadar pencatatan kerusakan (*Damage Control*), tetapi amunisi bagi Arsitek Solusi (*Solution Architect*) untuk mendesain ulang fondasi yang runtuh.

Jika tim forensik membuktikan bahwa kebocoran data bermula dari pengembang (*DevOps*) yang memasang secara permanen (*Hardcoding*) kunci akses Rahasia (*Secret AWS Key*) ke dalam kode sumber aplikasi Node.JS mereka, solusi pencegahan selanjutnya bukanlah memberi teguran lisan kepada *Developer*.

Solusi pencegahan berarsitektur tinggi (*Architectural Refactoring*) adalah membuang metode identitas kunci absolut tersebut secara permanen dari sistem (*Deprecation*), dan memigrasikan metode identifikasi (*Authentication*) Aplikasi tersebut menjadi berbasis *Pendelegasian Dinamis Tersementara* **(AWS IAM Roles for EC2 / Azure Managed Identities)**, di mana *Server Cloud* dan Aplikasi *Container* menegosiasikan token akses dinamis sesaat yang masa aktifnya (*Lifespan*) akan kadaluwarsa setiap 15 Menit secara sistemik. Kerentanan yang diperbaiki di struktur fondasi awan tak akan berulang.

## 5.13 Konklusi Investigasi Awan Modern

Komputasi Awan (*Cloud Computing*) telah mempercepat inovasi produk pengembangan perangkat lunak (*DevOps*) secara transformasional. Layanan rilis puluhan kali dalam hitungan detik. Paradigma keamanan (DFIR) tidak punya pilihan lain kecuali menyesuaikan *Gigi Transmisi Kecepatan* dengan irama orkestrasi pergerakan *Cloud-Native* tersebut, atau tertinggal dalam abu sejarah (*Obsolescence*).

Insinyur Forensik Awan (*Cloud Security Analysts*) modern bukanlah operator yang memutar kabel dan mengekstrak memori HDD. Mereka adalah ilmuwan data yang merajut kueri SQL lintas *Database Athena*, mereka adalah pembuat automasi Bot *AWS Lambda* untuk respons krisis milidetik, mereka adalah master bahasa identitas *IAM (JSON Policies)* yang menjadi urat nadi izin (*Permissions*) mesin modern.

Di alam *Cloud-Native* (AWS, Azure, K8s), identitas akses merupakan kunci, konfigurasi mikro adalah gembok, dan perbaikan berkelanjutan adalah gerbang perimeter satu-satunya. Kuasai visibilitas aliran *Control Plane* dan kuasai perburuan otomatis (*Automated Hunting*) untuk bertahan di ekosistem peretasan skala besar ini.

# GLOSARIUM CLOUD SECURITY (A-M)

| TERMINOLOGI | DEFINISI & KONSEP OPERASIONAL AWAN |
|---|---|
| **CloudTrail / Activity Log** | Log *Control Plane*. Mencatat setiap pemanggilan antarmuka pemrograman API (Siapa, Kapan, Aksi Apa) yang memodifikasi mesin/database di akun AWS/Azure Anda. |
| **CNAPP** | *Cloud-Native Application Protection Platform*. Paket solusi keamanan awan lengkap yang menggabungkan CSPM (Postur), CIEM (Identitas), dan CWPP (*Workload/Container*). |
| **Container Escape** | Eksploitasi di mana peretas yang telah menjebol aplikasi *Pod K8s* berhasil merusak *sandbox* (Dinding isolasi OS) dan menguasai *Server Node Host* di bawahnya. |
| **Ephemeral Workloads** | Infrastruktur komputasi sementara (Kontainer/Fungsi Nirserver) yang dapat lenyap, dihapus secara sistemik oleh orkestrasi awan (K8s/AWS Auto-Scale) dalam rentang kedipan mata. |
| **IAM / Entra ID** | *Identity and Access Management* (AWS) atau Entra ID (Microsoft Azure). Sarana kontrol pusat, pengelola identitas otentikasi, hak wewenang akses *User* dan izin aplikasi mesin. |

# GLOSARIUM CLOUD SECURITY (N-Z)

| TERMINOLOGI | DEFINISI & KONSEP OPERASIONAL AWAN |
|---|---|
| **Serverless (Nirserver)** | Mode komputasi awan (Contoh: AWS Lambda / Azure Functions) di mana perusahaan tidak mengelola peladen OS. Sistem akan menghidupkan/mematikan blok skrip seketika saat diakses. |
| **Shared Responsibility** | Model tanggungan yang menyatakan AWS/Azure mengamankan lapisan perangkat keras Fisik (Hipervisor), tetapi Pelanggan harus mutlak mengamankan Konfigurasi IAM dan Datanya. |
| **Shift-Left Security** | Paradigma pencegahan di mana arsitek memajukan perbaikan keamanan sedini mungkin pada fase perancangan kode perangkat lunak (*CI/CD Code Review*) sebelum diunggah ke *Cloud*. |
| **SOAR / Automation** | *Security Orchestration, Automation, and Response*. Kode respons bot (*Playbook*) yang mengeksekusi otomatisasi isolasi/pencabutan hak penyerang di *Cloud* tanpa manusia. |
| **VPC Flow Logs** | Log *Virtual Private Cloud* AWS (Mirip *NetFlow* jaringan). Merekam meta-telemetri koneksi pergerakan masuk dan keluar paket dari *Subnet/Server* untuk melacak eksfiltrasi. |



Di era komputasi awan, perlindungan perimeter telah mati. Identitas Anda (IAM) dan ketangkasan Anda merespons API adalah satu-satunya pelindung yang nyata.

# The Elastic Perimeter

© 2026 All rights reserved.
Cloud-Native & Response Architecture